EVIDENCIA DE ÍNDICES: EXPLAIN ANALYZE ANTES Y DESPUÉS

1. Objetivo

Evaluar el comportamiento de una consulta SQL antes y después de crear el índice idx_pedido_empleado_fecha, definido sobre las columnas id_empleado_registra y fecha_hora de la tabla pedido.

2. Justificación del índice

El índice se utiliza para apoyar consultas que buscan los pedidos registrados por un empleado y los ordenan por fecha y hora. Las columnas seleccionadas corresponden a un filtro frecuente y al orden cronológico de los resultados.

3. Consulta evaluada

La pregunta de negocio es: ¿qué pedidos registró un empleado específico y en qué orden cronológico?

```
EXPLAIN ANALYZE

SELECT id_pedido, id_empleado_registra, fecha_hora, estado

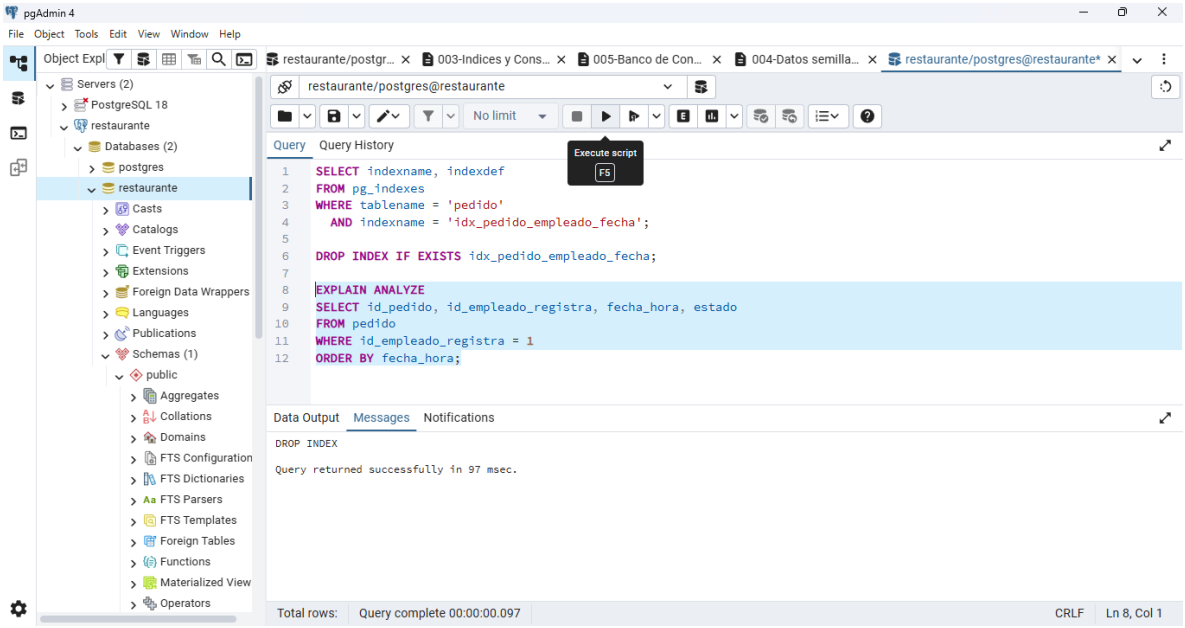
FROM pedido

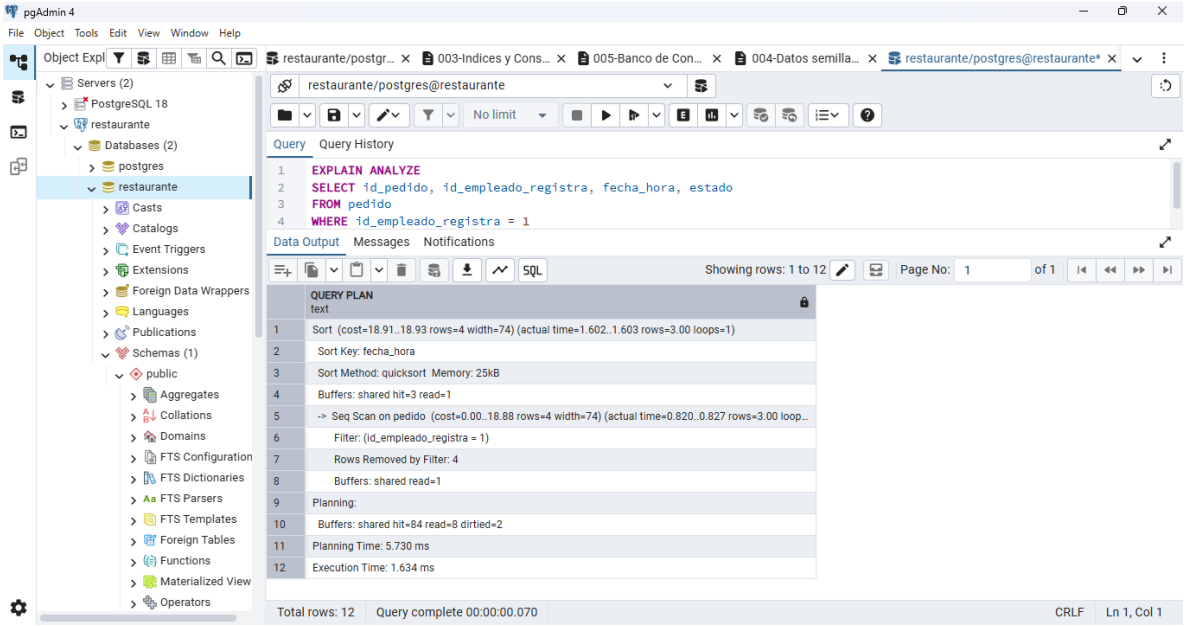
WHERE id_empleado_registra = 1

ORDER BY fecha_hora;
```

4. Plan de ejecución antes de crear el índice

Se eliminó temporalmente el índice seleccionado y se ejecutó la consulta anterior con EXPLAIN ANALYZE para observar el plan de ejecución sin ese índice.





Observaciones del plan anterior:

- Tipo de escaneo: Seq Scan.
- Filas observadas: 3 filas devueltas y 4 filas descartadas por el filtro.
- Tiempo de ejecución: 1.634 ms.

5. Índice creado

Se creó nuevamente el índice mediante el siguiente comando:

```
CREATE INDEX idx_pedido_empleado_fecha
ON pedido(id_empleado_registra, fecha_hora);
```

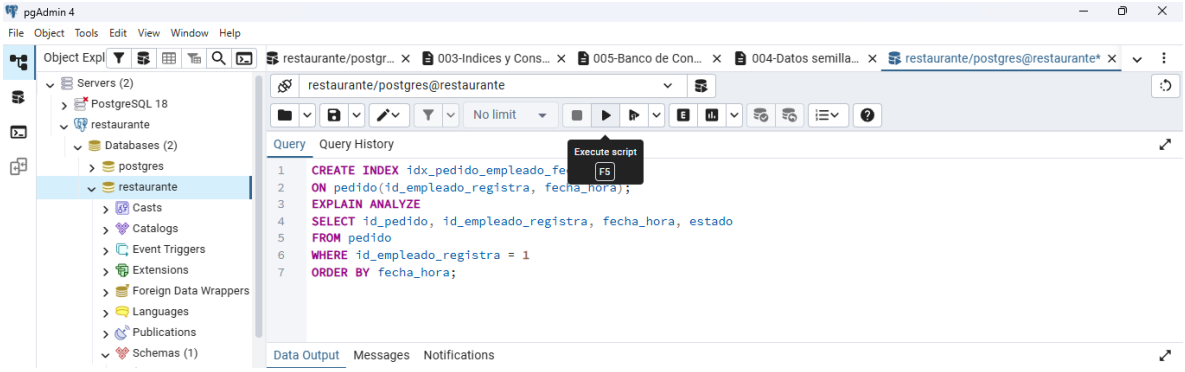
6. Plan de ejecución después de crear el índice

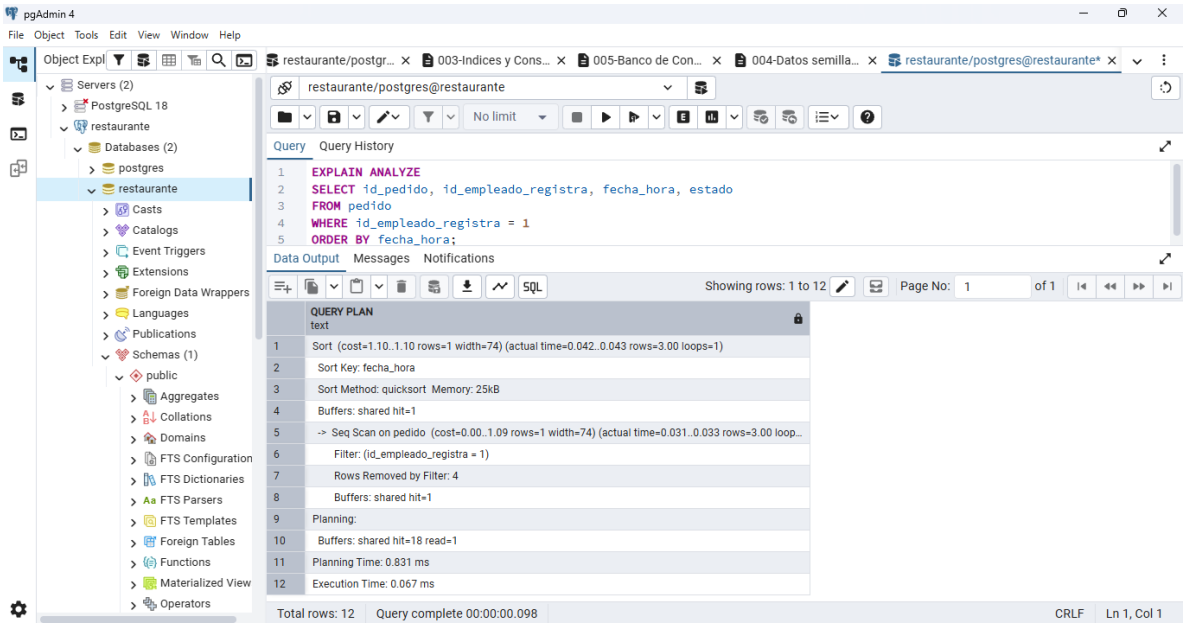
Después de crear el índice idx_pedido_empleado_fecha, se ejecutó nuevamente la misma consulta mediante EXPLAIN ANALYZE.

El plan obtenido mostró que PostgreSQL utilizó un escaneo secuencial (Seq Scan) sobre la tabla pedido, seguido de una operación de ordenamiento (Sort) por la columna fecha_hora. La consulta encontró 3 filas y descartó 4 que no cumplían el filtro aplicado.

El tiempo de ejecución reportado fue de 0.067 ms.

Aunque el índice fue creado correctamente, PostgreSQL no lo utilizó en esta ejecución. Esto puede explicarse por el reducido número de registros de la tabla, ya que recorrer una tabla pequeña puede resultar menos costoso que utilizar un índice.





Observaciones del plan posterior:

- Tipo de escaneo: Seq Scan.
- Filas observadas: 3 filas encontradas y 4 filas descartadas por el filtro.
- Tiempo de ejecución: 0.067 ms.

7. Comparación de resultados

Aspecto	Antes	Después
Tipo de escaneo	Seq Scan	Seq Scan
Tiempo de ejecución	1.634 ms	0.067 ms
Uso del índice	No	No
Filas devueltas	4	4
Filas descartadas	3	3

8. Conclusión

Al comparar los planes de ejecución antes y después de crear el índice idx_pedido_empleado_fecha, se observó que PostgreSQL utilizó un escaneo secuencial (Seq Scan) en ambas pruebas, por lo que no hizo uso del índice creado.

Antes de crear el índice, el tiempo de ejecución fue de **1.634 ms**, mientras que después fue de **0.067 ms**. Aunque se presentó una disminución en el tiempo medido, no se puede afirmar que esta mejora haya sido causada por el índice, debido a que PostgreSQL no lo utilizó en ninguno de los dos planes.

Esto puede deberse a que la tabla pedido contiene pocos registros, por lo que el optimizador considera más conveniente recorrerla directamente. En conclusión, la prueba permitió verificar el comportamiento de la consulta mediante EXPLAIN ANALYZE y evidenciar que la creación de un índice no garantiza que PostgreSQL lo utilice en todas las consultas.

9. Índices para claves foráneas y consultas frecuentes

En la migración 003_indices.sql se definieron índices para apoyar las consultas frecuentes y la búsqueda de registros relacionados entre las tablas del sistema.

Los índices creados son:

Índice	Tabla y columnas	Propósito
idx_empleado_id_cargo	empleado(id_cargo)	Buscar empleados según su cargo.
idx_empleado_telefono_empleado	empleado_telefono(id_empleado)	Consultar los teléfonos asociados a un empleado.
idx_reserva_cliente_fecha	reserva(id_cliente, fecha)	Consultar las reservas de un cliente por fecha.
idx_reserva_mesa_fecha_hora	reserva(id_mesa, fecha, hora, hora_fin)	Buscar reservas asociadas a una mesa y revisar sus horarios.
idx_producto_categoria_disponible	producto(id_categoria, disponible)	Filtrar productos por categoría y disponibilidad.
idx_producto_ingrediente_ingrediente	producto_ingrediente(id_ingrediente)	Buscar productos relacionados con un ingrediente.
idx_pedido_cliente_fecha	pedido(id_cliente, fecha_hora)	Consultar los pedidos de un cliente por fecha y hora.
idx_pedido_mesa_fecha	pedido(id_mesa, fecha_hora)	Consultar los pedidos asociados a una mesa por fecha y hora.
idx_pedido_empleado_fecha	pedido(id_empleado_registra, fecha_hora)	Consultar los pedidos

		registrados por un empleado en orden cronológico.
idx_pedido_estado_fecha	pedido(estado, fecha_hora)	Filtrar pedidos por estado y fecha.
idx_item_producto	item_pedido(id_producto)	Buscar los elementos de los pedidos correspondientes a un producto.
idx_atencion_pedido	atencion(id_pedido)	Consultar las atenciones relacionadas con un pedido.
idx_prepara_pedido	prepara(id_pedido)	Consultar las preparaciones relacionadas con un pedido.
idx_atencion_empleado	atencion(id_empleado)	Consultar las atenciones registradas por un empleado.

Los índices se definieron en una migración independiente para facilitar su ejecución y mantenimiento. Su selección se relaciona con las columnas utilizadas en filtros, búsquedas y consultas que conectan información de diferentes tablas.

La evidencia de EXPLAIN ANALYZE se realizó específicamente con el índice idx_pedido_empleado_fecha. Los demás índices se encuentran definidos en la migración 003_indices.sql, su utilidad depende de las consultas ejecutadas, la cantidad de registros y la decisión del optimizador de PostgreSQL.

10. Verificación final

Al terminar la medición, se comprobó que el índice idx_pedido_empleado_fecha quedó creado nuevamente en la base de datos.